

MOVIE BOOKING APP

IIIHT

CONTENTS

1	Problem Statement	1
2	Proposed Movie Booking App Wireframe	1
3	Application Architecture	1
4	Tool Chain	1
5	Development flow	2
6	Business-Requirement:	3
7	Rubrics/Expected Deliverables	4
7.1	Rest API (Products & Frameworks -> Compute & Integration):	4
7.2	Database (Products & Frameworks -> Database & Storage):	5
7.3	Maven (Tooling):	5
7.4	Unit Testing:	5
7.5	Debugging & Troubleshooting	5
7.6	Code Quality & Code Coverage:	5
7.7	Good To Have:	5
7.8	Expected Outcomes:	5
7.9	List All movies:	5
7.10	Search By Movie Name:	5
7.11	User Registration:	5
7.12	First must login to Book a Ticket:	5
7.13	Login:	5
7.14	Logged in User:	5
7.15	Ticket Booked	5
7.16	Get a Count of Ticket Booked:	5
7.17	Get Tickets booked by User	5
7.18	Admin Added a new movie:	5

1 PROBLEM STATEMENT

Movie Booking App is an application that allows the users to register and login and search movies released. User can book the tickets for the movies they wish for. Admin shall view the tickets booked and update the pending tickets available to the system.

Guest users cannot book any tickets.

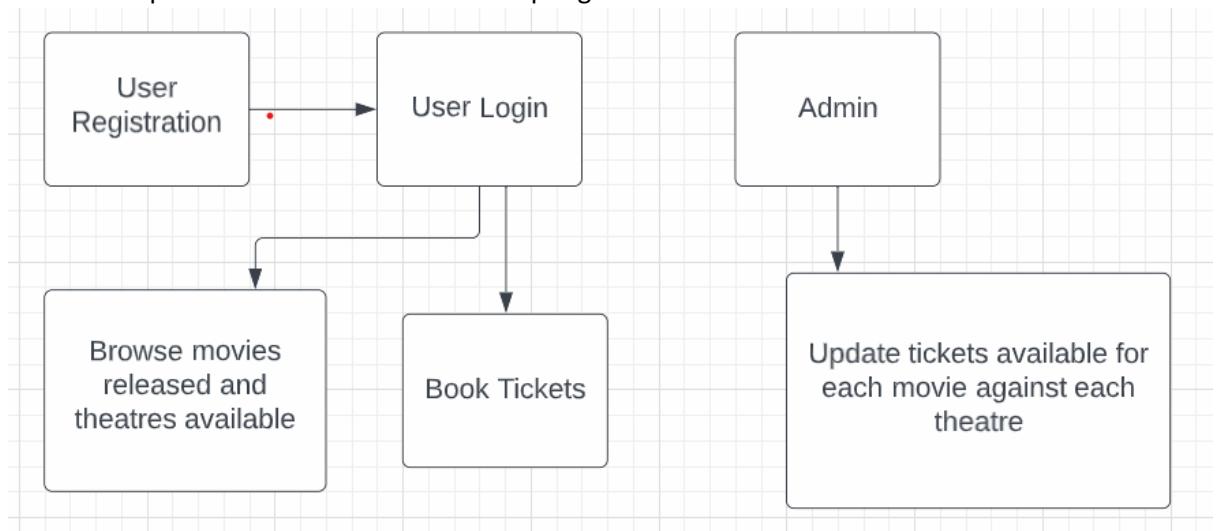
The core modules of movie booking app are:

1. User registration and login
2. Browse the movies released and list of theatres
3. User can book the tickets
4. Admin to update the available ticket status based on the tickets booked

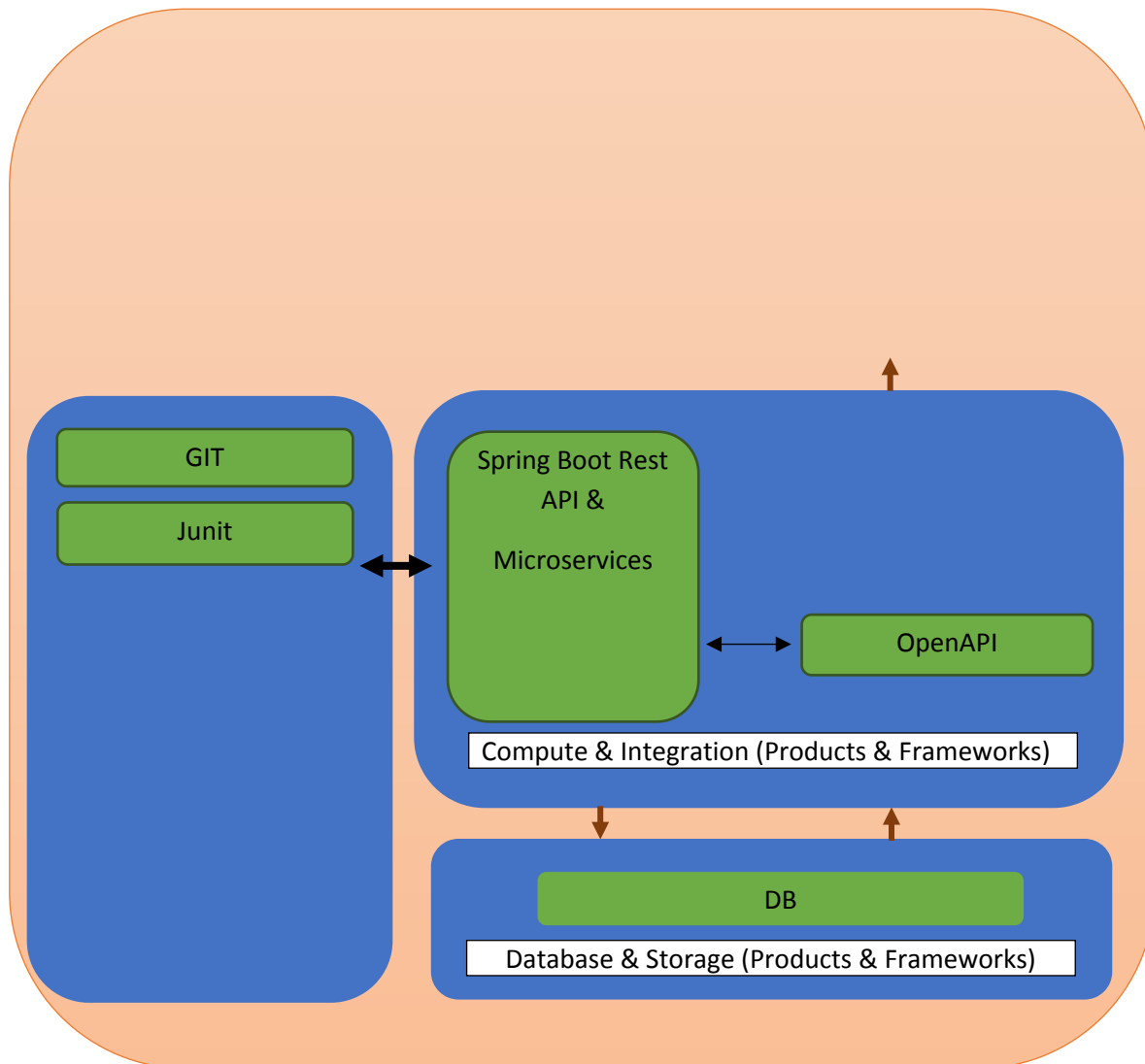
The scope includes developing the application using toolchain mentioned below.

2 PROPOSED MOVIE BOOKING APP WIREFRAME

1. UI needs improvisation and modification as per given use case.



3 APPLICATION ARCHITECTURE



4 TOOL CHAIN

Competency	Skill	Skill Detail
Engineering Mindset	Networking and Content Delivery	
	Ways of Working	
	Consulting Mindset	
	DevOps	
Programming Languages	Application Language	Java
Products & Frameworks	Presentation	
	Compute & Integration	Spring Boot
	Database & Storage	MySQL / SQL-Server
	Governance & Tooling	Git
		Maven
		Junit 5 / Mockito

5 DEVELOPMENT FLOW

No	MC	Competency	Section	Indicative Mechanism for Evaluation (Passing score of 60% in each MC)
Business Requirement				
1	Backend	Rest API, Database, Unit Testing, Debugging & Troubleshooting	Click here	Code Submission and Evaluation, Panel Presentation

6 BUSINESS-REQUIREMENT:

As an application developer, develop frontend, middleware and deploy the Movie Booking App with below guidelines:

Pre-requisite before implementing the user story:

Create a static database as “Movie” and add the below fields

- Movie name, Total number of tickets allotted and theatre name (Movie name and theatre name should be of composite primary key)

There should be a predefined set of tickets assigned to each theatre against each movie. Just create sample 2 movies with 2 theatres assigned. This will be used for the upcoming user story to fetch the data.

User Story #	User Story Name	User Story
US_01	Registration and Login	As a user I should be able to login/Register in the movie booking application Acceptance criteria: 1. A logged-in user can reset their password so they can login, even if they forget their password. 2. A logged-in user: a. Cannot change their username. b. Can logout from their account. 3. As a customer I should be able to furnish following details at the time of registration a. First Name b. Last Name c. Email d. Login Id e. Password f. Confirm Password g. Contact Number 4. All details fields must be mandatory 5. Login Id and Email must be unique 6. Password and Confirm Password must be same 7. If any constraint is not satisfied, validation message must be shown
US_02	View &Search Movies	As a user I should be able to view all the recent movies opened for booking. User can search any particular movies as well Acceptance criteria: a. User can view all the existing released movies. b. User can search any particular movie based on the movie name
US_03	Book Tickets for a movie	As a user I should be able to book tickets for a movie. Add this booking to a database table "Tickets". Assign movie name and theatre name as a foreign key to database table "Movie" which is already created as a pre-requisite Acceptance criteria: a. Book a movie ticket b. Below are the details to be added • Movie Name • Theatre name • Number of tickets • Seat Number

US_04	View booked tickets and Update ticket status	As an admin I should be able to view booked tickets and update the balance tickets available for a movie Acceptance criteria: - View number of booked tickets for a particular movie from table "Tickets" - Check the tickets available from table "Movie" by calculating the total tickets booked - If the quantity is 0, update the ticket status as 'SOLD OUT', else update as 'BOOK ASAP'
-------	--	---

7 RUBRICS/EXPECTED DELIVERABLES

7.1 REST API (PRODUCTS & FRAMEWORKS -> COMPUTE & INTEGRATION):

- Use Spring Boot to version and implement the REST endpoints.
- Implement HTTP methods like GET, POST, PUT, DELETE, PATCH to implement RESTful resources:

POST	/api/v1.0/moviebooking/register	Register as new user
GET	/api/v1.0/ moviebooking /login	Login
GET	/api/v1.0/ moviebooking /<username>/forgot	Forgot password
GET	/api/v1.0/ moviebooking /all	View all movies
GET	/api/v1.0/moviebooking/movies/search/moviename *	Search by movie name
POST	/api/v1.0/ moviebooking /<moviename>/add	Book tickets for a movie
PUT	/api/v1.0/moviebooking /<moviename>/update/<ticket>	Update ticket status
DELETE	/api/v1.0/ moviebooking /<moviename>/delete/<id>	Delete movie

- *Movie name may be partial or complete username
- Use necessary configuration in place for REST API in application. Properties or bootstrap. Properties or application.yml; whichever is applicable.
- Package Structure for Spring Boot Project will be like com. moviebookingapp. * With proper naming conventions for package and beans.
- Use configuration class annotated with @Configuration and @Service for business layer.
- Use constructor-based dependency injection in few classes and setter-based dependency injection in few classes.
- Follow Spring Bean Naming Conventions
- Implement SQL procedures, triggers, and other performance-enhancing techniques.
- Implement at least 2 role-based authentications/authorisations in your Spring Boot applications.

- k. Implement cache and session management techniques using the Spring Boot API.

7.2 DATABASE (PRODUCTS & FRAMEWORKS -> DATABASE & STORAGE):

1. As an application developer:
 - a. Implement ORM with Spring Data MongoRepository and MongoDB/ JpaRepository for other databases like SQL server/ MySQL. For complex and custom queries, create custom methods and use @Query, Aggregations (Aggregation Operation, Match Operation, Aggregation Results), implementation of MongoTemplate etc as necessary.
 - b. Have the necessary configuration in place for REST API in application.properties or bootstrap.properties or application.yml OR Java based configuration; whichever is applicable.

7.3 MAVEN (TOOLING):

1. As an application developer:
 - a. Create the spring boot project using Maven CLI
 - b. Generate Surefire test reports and share it as a part of deliverables
 - c. Using Maven CLI generate the project documentation, and share it as a part of deliverables

7.4 UNIT TESTING:

1. Implement Unit testing using JUnit 5 for all the layers Exp: Controller layer, Service layer, and Repository layer.
2. Generate the test results for the positive and negative test scripts.

7.5 DEBUGGING & TROUBLESHOOTING

1. Generate bug report & error logs - Report must be linked with final deliverables which should also suggest the resolution for the encountered bugs and errors.

7.6 CODE QUALITY & CODE COVERAGE:

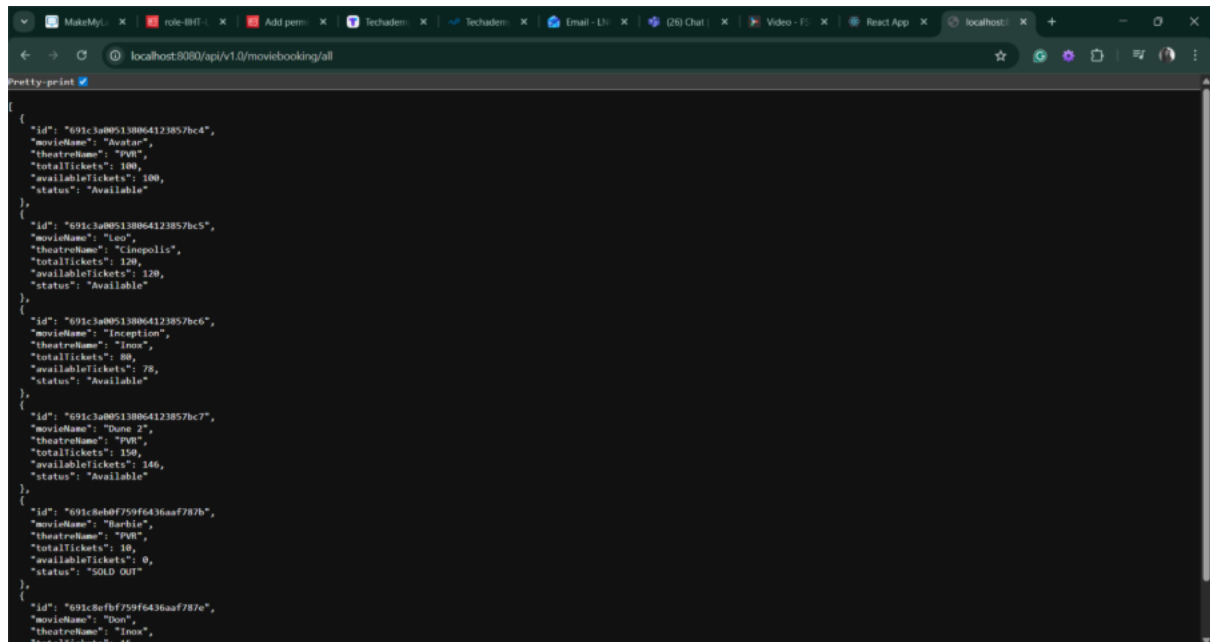
1. Ensure code quality with static analysis tools.
2. Code coverage for API should be > 80%

7.7 GOOD TO HAVE:

1. Implement error/exception handling mechanisms
2. Implement logging and monitoring to detect and troubleshoot issues
3. Implement rate limiting to prevent abuse of API endpoints
4. Blogs should be fetched and displayed within 30 seconds
5. Use containerization to package and deploy the application
6. Improve maintainability with modular codebase, automated testing, CI/CD pipelines

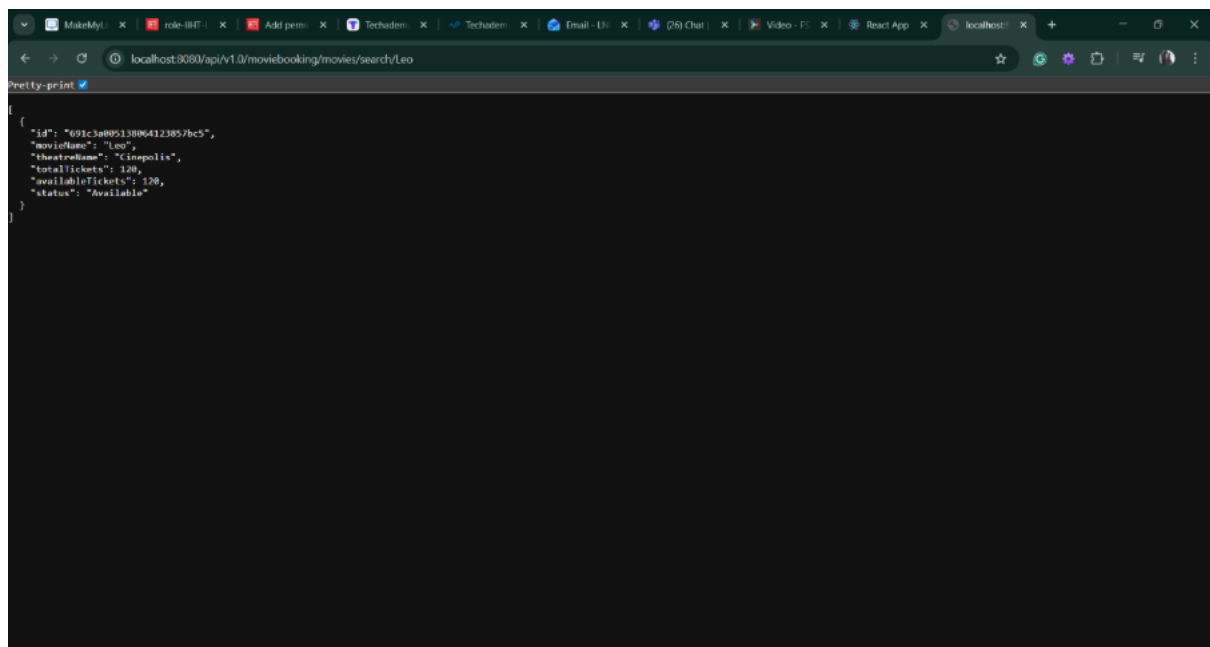
7.8 EXPECTED OUTCOMES:

7.9 List All movies:



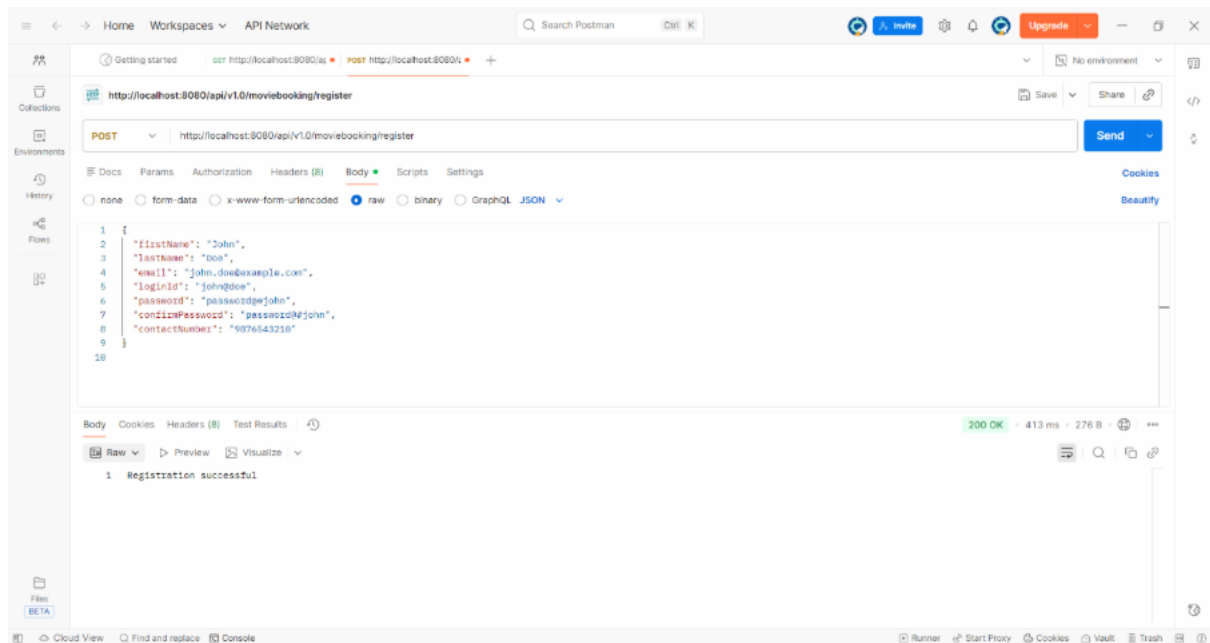
```
localhost:8080/api/v1.0/moviebooking/all
Pretty-print
{
  "id": "691c3a005138064123857bc4",
  "movieName": "Avatar",
  "theatreName": "PVR",
  "totalTickets": 100,
  "availableTickets": 100,
  "status": "Available"
},
{
  "id": "691c3a005138064123857bc5",
  "movieName": "Leo",
  "theatreName": "Cinepolis",
  "totalTickets": 120,
  "availableTickets": 120,
  "status": "Available"
},
{
  "id": "691c3a005138064123857bc6",
  "movieName": "Inception",
  "theatreName": "Inox",
  "totalTickets": 80,
  "availableTickets": 78,
  "status": "Available"
},
{
  "id": "691c3a005138064123857bc7",
  "movieName": "Don 2",
  "theatreName": "PVR",
  "totalTickets": 150,
  "availableTickets": 146,
  "status": "Available"
},
{
  "id": "691c8ebbf759f6436aaf787b",
  "movieName": "Barbie",
  "theatreName": "PVR",
  "totalTickets": 10,
  "availableTickets": 0,
  "status": "SOLD OUT"
},
{
  "id": "691c8ebbf759f6436aaf787e",
  "movieName": "Don",
  "theatreName": "Inox",
  "totalTickets": 15
}
```

7.10 Search By Movie Name:

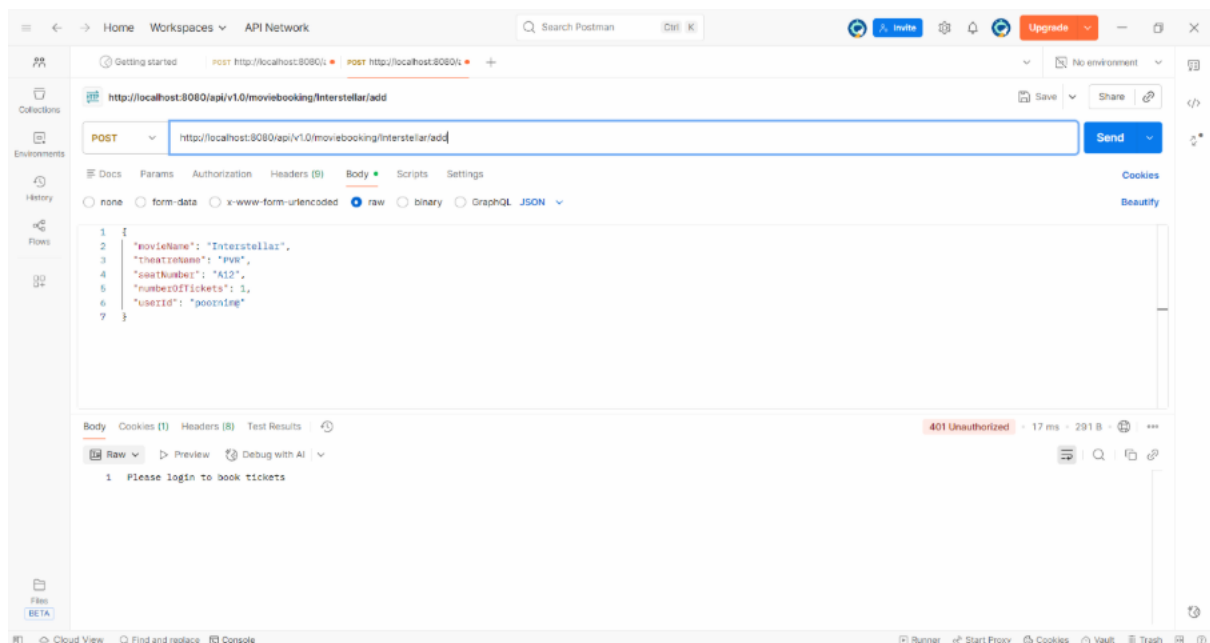


```
localhost:8080/api/v1.0/moviebooking/movies/search/Leo
Pretty-print
{
  "id": "691c3a005138064123857bc5",
  "movieName": "Leo",
  "theatreName": "Cinepolis",
  "totalTickets": 120,
  "availableTickets": 120,
  "status": "Available"
}
```

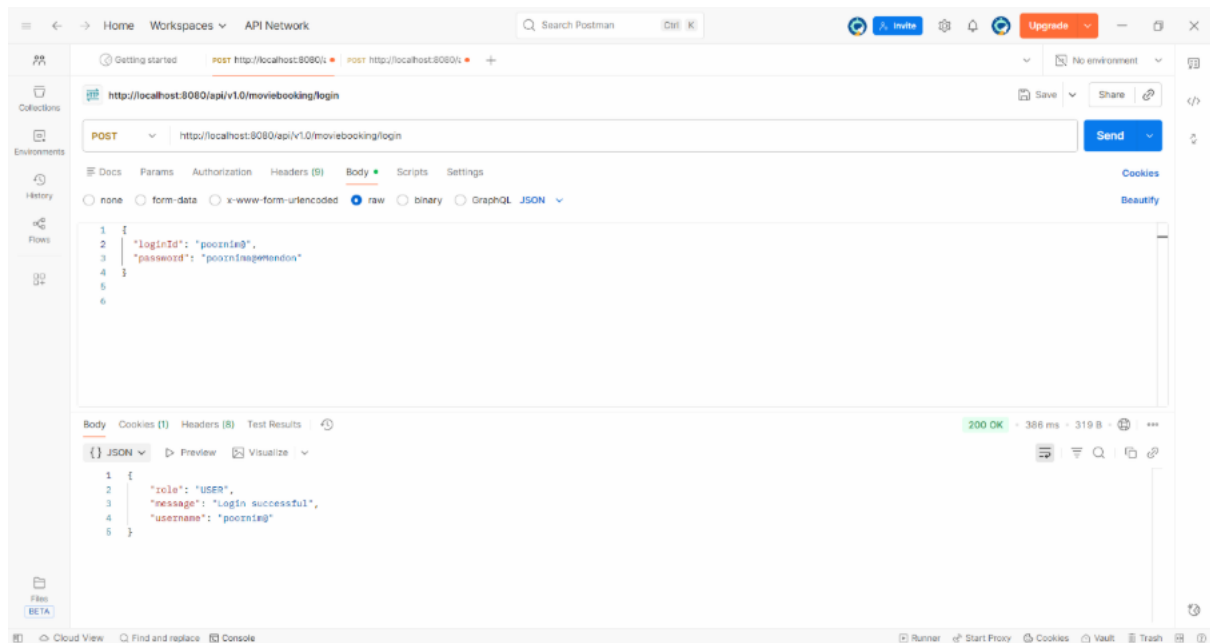
7.11 User Registration:



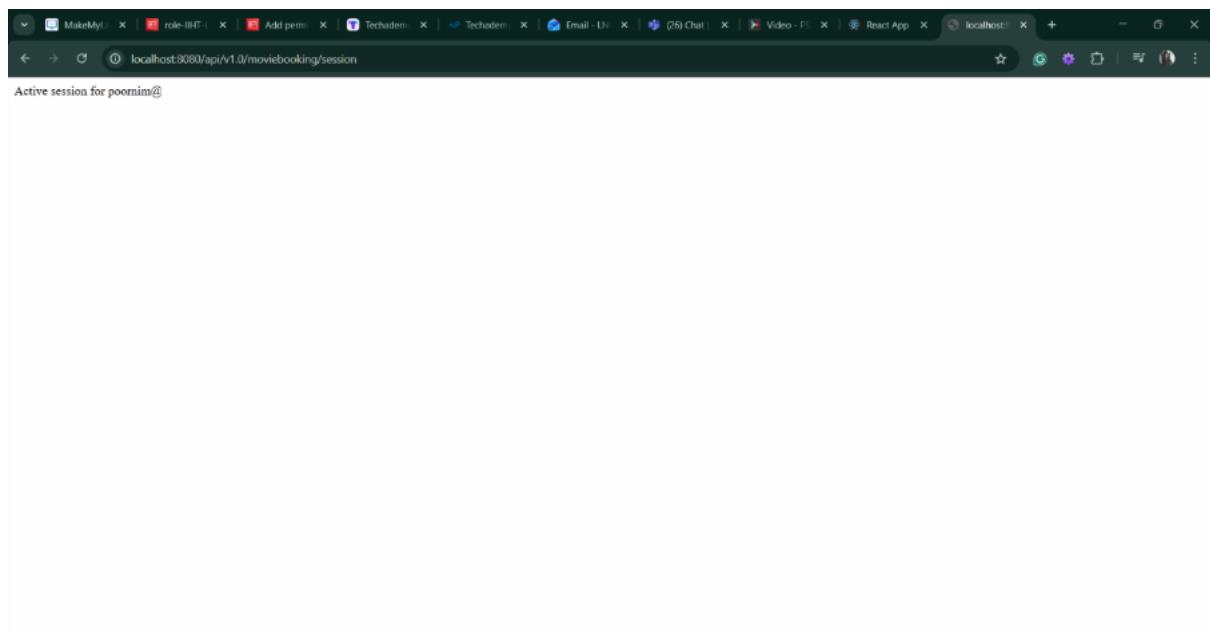
7.12 First must login to Book a Ticket:



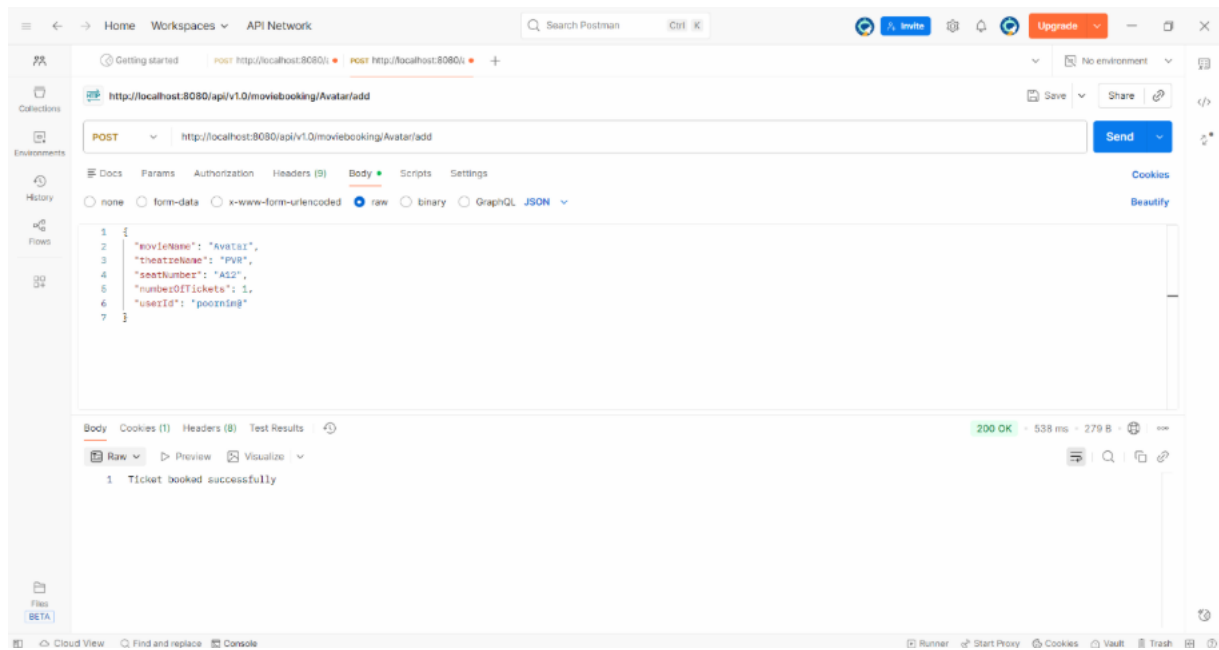
7.13 Login:



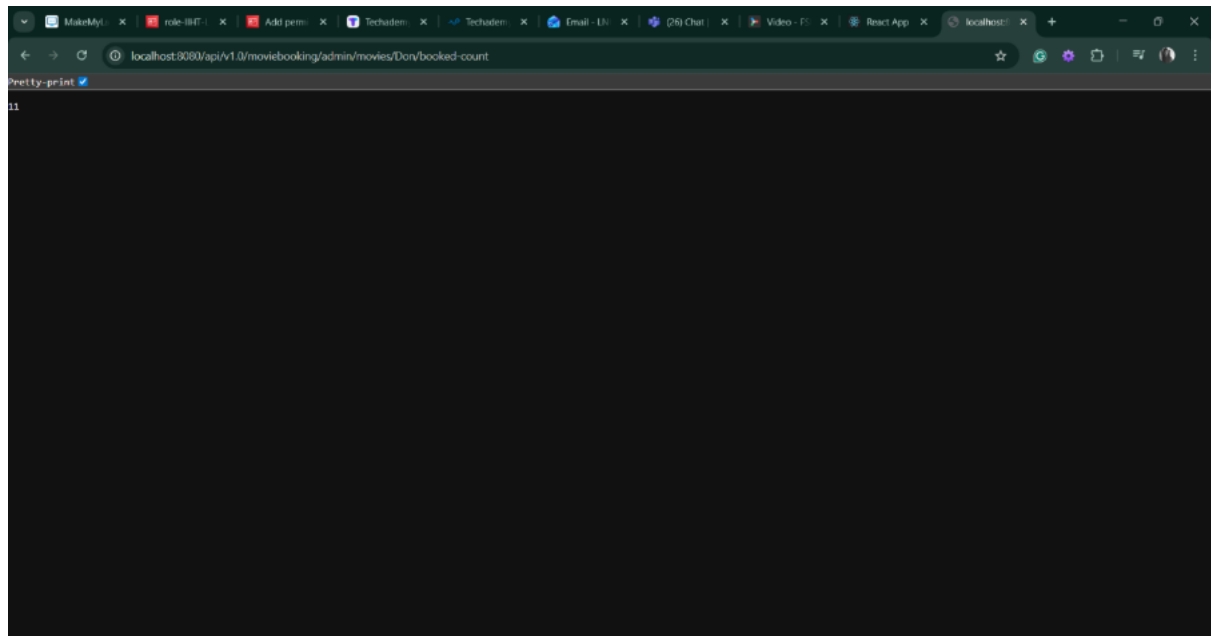
7.14 Logged in User:



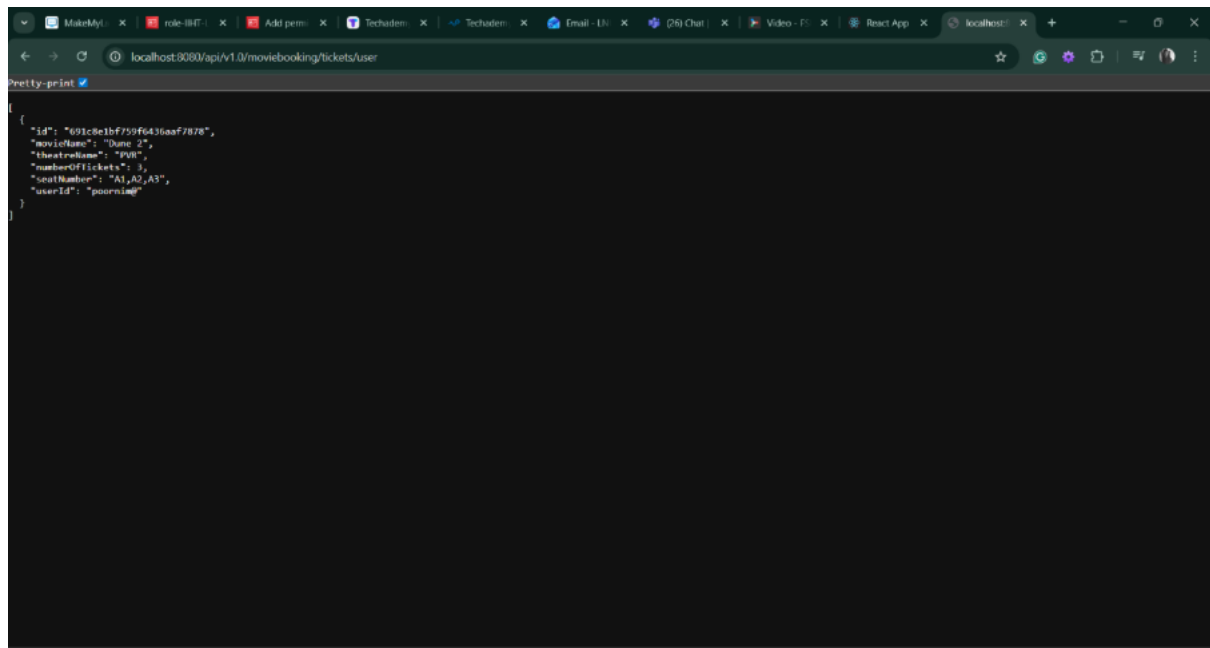
7.15 Ticket Booked



7.16 Get a Count of Ticket Booked:



7.17 Get Tickets booked by User



7.18 Admin Added a new movie:

